

Milestone 4 Report

Multiple-Choice QA with BERT: Tokenization, Multiple-Choice Modeling, and LoRA Fine-Tuning

Source notebook: milestone-4.ipynb

1. Overview

This notebook works through a multiple-choice question answering pipeline built on **bert-base-uncased**. It progresses in ten labeled steps (Q1–Q10): loading and label-encoding a science MCQ dataset, formatting prompt/option pairs for a transformer, tokenizing single and batched examples, running a **BertForMultipleChoice** model for inference and loss computation, applying **LoRA** (via PEFT) for parameter-efficient fine-tuning, wrapping tokenized examples in a Hugging Face **Dataset**, running a short **Trainer** loop, and finally performing inference with the fine-tuned model.

2. Dataset

The training data (`train.csv`) contains an `id`, a `prompt` (question text), five answer options (A–E), and the correct answer letter. Questions span topics such as philosophy (Heidegger) and physics (light-ion fusion, relativity of simultaneity). Labels are mapped from letters A–E to integers 0–4 for model training.

3. Step-by-Step Walkthrough

The notebook is organized into 10 questions, each building on the previous. A summary of what each step does and what it found:

Step	Task	Result
Q1	Inspect an encoded example	Verified the label mapping: answer 'C' at row 150 encodes to label 2.
Q2	Format prompt + option	Concatenated a prompt and option B with a [SEP] token; resulting string was 407 characters long.
Q3	Tokenize one question's 5 choices	Tokenized all 5 options with padding/truncation to length 128; <code>input_ids</code> shape (1, 5, 128) — one question, 5 choices, 128 tokens each.
Q4	Batch tokenization	Tokenized 16 questions (5 choices each); resulting batch shape (16, 5, 128), totaling 10,240 token positions.
Q5	Multiple-choice forward pass	Ran <code>BertForMultipleChoice</code> on one question; logits shape (1, 5) — one score per option, before fine-tuning (near-uniform, ~0.10-0.12).
Q6	Compute loss	Passed the true label into the model to get a scalar cross-entropy loss (~1.59) with 0 dimensions (<code>ndim=0</code>).
Q7	Apply LoRA	Attached a LoRA adapter (<code>r=8</code> , <code>alpha=16</code>) to the query/value projections. Only 295,681 of 109,778,690 parameters (0.27%) are trainable.

Step	Task	Result
Q8	Build a HF Dataset	Tokenized the first 100 rows into a Hugging Face Dataset with input_ids/attention_mask/labels fields (5 x 128 per example).
Q9	Train with Trainer	Built a 32-example dataset (max_length=64), reloaded BertForMultipleChoice + LoRA, and ran a short Trainer session (final global_step = 4).
Q10	Inference after training	Ran the fine-tuned model on question 0 in eval mode; softmax probabilities were still close to uniform (~0.20 each), with Option E at 0.1985.

4. Key Technical Details

4.1 Input formatting

Each option is formatted as prompt [SEP] option_text, giving the model both the question and one candidate answer per input. For a multiple-choice question with 5 options, this produces 5 separate sequences that BERT scores independently, then compares.

4.2 Tensor shapes

The multiple-choice input convention is (batch_size, num_choices, seq_len). For one question this is (1, 5, 128); for a batch of 16 questions it becomes (16, 5, 128). The model's output logits collapse the choice dimension into a single score per option, giving shape (batch_size, num_choices) — e.g. (1, 5).

4.3 LoRA fine-tuning

Rather than updating all ~110M BERT parameters, LoRA injects small trainable low-rank matrices into the query and value projection layers (rank r=8, scaling alpha=16, dropout 0.1). This drops the trainable parameter count to about 0.27% of the full model, making fine-tuning far cheaper while leaving the pretrained weights frozen.

4.4 Training and inference

A Hugging Face Trainer was run on a small 32-example subset for a handful of steps (global_step = 4) — enough to validate the pipeline runs end-to-end, but not enough data or steps to meaningfully learn. This shows in the final inference: predicted probabilities across the 5 options stayed close to uniform (~0.20 each), consistent with a model that hasn't yet learned to discriminate between correct and incorrect options.

5. Observations & Next Steps

- The pipeline is correctly wired end-to-end: data loading → formatting → tokenization → multiple-choice modeling → LoRA → Dataset → Trainer → inference.
- The near-uniform post-training probabilities (Q10) indicate the model has not meaningfully learned yet — expected given only 32 training examples and 4 steps.
- Scaling up would mean: training on the full dataset (or a much larger subset), more epochs/steps, and likely a learning-rate/batch-size sweep once the short run confirms the setup works.
- A validation split and an accuracy/MAP@3-style metric would be needed to actually measure whether fine-tuning is improving option ranking, since loss and raw probabilities alone don't show that directly.